Sprint 3

Chris Lee, Jaimin Fee, Kurt Schimmel (Team 6)

WhosDown: Mobile app that makes pickup sports easier

1. API Assessment

All WhosDown data (games, RSVPs, profiles, friends, groups, messages, reviews, notifications) is created by our own users. We do not need data from an outside provider for it, and Firebase gives us authentication, a real-time database, file storage, and push messaging without running our own server. For these reasons we use **Firebase** as our backend and **do not build a custom API**.

Two outside services supply the data that users do not create:

Need	Service	Why
Accounts, sign-in, guest browsing	Firebase Authentication (email/ password + anonymous)	Built-in guest mode; anonymous accounts can be upgraded to email accounts later
App data	Cloud Firestore	Real-time listeners keep rosters, RSVP status, and chat up to date without polling
Profile photos, game photos and clips, group photos	Firebase Storage	Files are stored in Storage; Firestore keeps only the download URL
Push notifications	Firebase Cloud Messaging (FCM)	Approvals, game changes, cancellations, waitlist spots, messages
Map, place search, user location, distance	Apple MapKit + Core Location	Native on iOS, no API key, no cost. Replaces the Google Maps plan from Sprint 1
Weather for outdoor games	Open-Meteo forecast API	Free, no API key. Answers our users' "will the field be playable?" concern (Section 4)
Add game to calendar	EventKit (on device)	Writes the game to the user's calendar

Change since Sprint 1: we replaced the Google Maps SDK and Places API with Apple MapKit (`Map`, `MKLocalSearch`). MapKit covers map pins, place search, and distance, needs no key or billing account, and fits SwiftUI directly.

2. Firebase JSON Structure and Setup

2.1 Collection tree

```
users/{userId}                          public profile + stats
    private/settings                    owner-only settings (one document)
    friends/{friendId}                  friend list and friend requests
    notifications/{notificationId}      in-app notification inbox
venues/{venueId}                        fields, courts, gyms
games/{gameId}                          pickup games
    rsvps/{userId}                      one per player (pending, approved, waitlisted, ...)
groups/{groupId}                        play groups / clubs
    members/{userId}                    group membership
conversations/{conversationId}          direct, game, and group chats
    messages/{messageId}                chat messages
reviews/{reviewId}                      post-game player reviews
reports/{reportId}                      safety reports
```

Design notes

- **Denormalized fields.** Values that many screens show (`hostName`, `venueName`, `joinedCount`, `attendeePreview`, `displayName` on RSVPs) are copied into the parent document. A list screen then needs one query, not one query per row.
- **RSVP document ID = user ID.** A user can have only one RSVP per game, and the app can read `games/{gameId}/rsvps/{me}` directly.
- **Counters** (`joinedCount`, `pendingCount`, `waitlistCount`, `memberCount`) are changed in the same Firestore transaction as the RSVP or member write, so they always match.
- **Location search.** Firestore has no radius query, so each game and venue stores a `geohash`. The app queries the geohash ranges that cover the search radius, then filters by exact distance on the device.
- **Text search.** Firestore has no full-text search, so `titleLowercase`, `nameLowercase`, and `displayNameLowercase` support prefix search on the Search screen (H48).
- **Private data.** Email, notification preferences, privacy settings, blocked users, and push tokens are in `users/{id}/private/settings`, which only the owner can read.

In the examples below, Firestore `Timestamp` values are written as ISO 8601 strings and `GeoPoint` values as `{latitude, longitude}`. Every collection has at least one populated record.

2.2 Documents

`users/{userId}` — public profile

```
{
  "displayName": "Jamie Morales",
  "displayNameLowercase": "jamie morales",
  "photoURL": null,
  "bio": "Midfielder. I organize weeknight games and keep them welcoming but organized.",
  "homeArea": "Oakland / CMU",
  "schoolLine": "CMU '27",
  "position": "Midfielder",
  "isVerified": true,
  "isGuest": false,
  "playStyles": ["social", "competitive"],
  "sportLevels": { "soccer": "intermediate", "tennis": "casual" },
  "stats": {
    "gamesRSVPd": 32,
    "gamesAttended": 31,
    "gamesHosted": 14,
    "mvpCount": 2,
    "playAgainYes": 16,
    "playAgainTotal": 18,
    "feedbackTagCounts": { "reliable": 12, "goodSport": 9, "communicative": 7 },
    "recentFeedback": ["Clear communication and always shows up."],
    "avgResponseMinutes": 20
  },
  "createdAt": "2026-09-01T18:22:00Z"
}
```

`users/{userId}/private/settings` — readable and writable by the owner only

```
{
  "email": "alex.morgan@example.com",
  "notificationPrefs": {
    "approvals": true,
    "timeOrLocationChanges": true,
    "cancellations": true,
    "reviewReminders": true,
```

```
    "newNearbyGames": false,
    "groupActivity": false,
    "messages": true
  },
  "newGameAlert": {
    "sports": ["soccer", "tennis"],
    "radiusMiles": 3.0,
    "weekdaysOnly": true,
    "earliestHour": 17,
    "skillLevels": ["allLevels"]
  },
  "privacy": {
    "gameActivity": "friends",
    "profileVisibility": "whosDownPlayers",
    "directMessages": "friendsAndTeammates",
    "friendRequests": "everyone"
  },
  "blockedUserIds": ["u_83jd"],
  "favoriteVenueIds": ["v_cmu_soccer", "v_schenley_oval"],
  "pinnedGroupIds": ["g_westside"],
  "searchRadiusMiles": 3.0,
  "fcmTokens": ["<device push token>"]
}
```

users/{userId}/friends/{friendId} — `status` is `requestSent`, `requestReceived`, or `friends`

```
{
  "displayName": "Mina Kim",
  "status": "friends",
  "mutualCount": 4,
  "since": "2026-10-02T23:10:00Z"
}
```

users/{userId}/notifications/{notificationId}

```
{
  "type": "gameUpdated",
  "title": "Pickup Soccer changed location",
  "body": "Jamie: \"Lights are better on the lower turf.\"",
  "gameId": "gm_7d2k",
  "groupId": null,
  "conversationId": null,
  "change": {
    "field": "location",
    "oldValue": "CMU Upper Turf",
    "newValue": "CMU Lower Turf",
    "hostNote": "Lights are better on the lower turf."
  },
  "expiresAt": null,
  "isRead": false,
  "createdAt": "2026-10-28T22:41:00Z"
}
```

venues/{venueId}

```
{
  "name": "CMU Soccer Field",
  "address": "5000 Forbes Ave, Pittsburgh, PA 15213",
  "location": { "latitude": 40.4433, "longitude": -79.9436 },
```

```
  "geohash": "dppnhf",
  "isIndoor": false,
  "surface": "turf",
  "hasLights": true,
  "sports": ["soccer", "football", "ultimate"],
  "createdBy": "u_jamie"
}
```

games/{gameId}

```
{
  "title": "Pickup Soccer",
  "titleLowercase": "pickup soccer",
  "sport": "soccer",
  "customSportName": null,
  "description": "Friendly all-level pickup.",
  "notes": "Bring a light + dark shirt. Need 1 goalkeeper.",
  "hostId": "u_jamie",
  "hostName": "Jamie Morales",
  "groupId": null,
  "venueId": "v_cmu_soccer",
  "venueName": "CMU Soccer Field",
  "location": { "latitude": 40.4433, "longitude": -79.9436 },
  "geohash": "dppnhf",
  "isIndoor": false,
  "surface": "turf",
  "hasLights": true,
  "startTime": "2026-10-28T23:00:00Z",
  "endTime": "2026-10-29T00:30:00Z",
  "skillLevel": "allLevels",
  "visibility": "public",
  "joinType": "approvalRequired",
  "priceCents": 0,
  "playerLimit": 14,
  "waitlistEnabled": true,
  "joinedCount": 12,
  "pendingCount": 2,
  "waitlistCount": 0,
  "attendeePreview": ["AM", "MK", "JB", "TP", "CL", "RS"],
  "fieldReserved": true,
  "reservationWindow": "6:45–8:45 PM",
  "facilityNote": "No field conflict.",
  "mediaURLs": [],
  "status": "scheduled",
  "cancelReason": null,
  "conversationId": "c_gm_7d2k",
  "createdAt": "2026-10-24T15:02:00Z",
  "updatedAt": "2026-10-25T21:40:00Z"
}
```

games/{gameId}/rsvps/{userId} — `status` is `pending`, `approved`, `declined`, `waitlisted`, `left`, or `removed`

```
{
  "userId": "u_taylor",
  "displayName": "Taylor Ross",
  "reliability": 0.88,
  "skillLevel": "beginner",
  "status": "pending",
```

```
  "isHost": false,
  "noteToHost": "Happy to play defense or learn in goal.",
  "hostResponse": null,
  "waitlistPosition": null,
  "spotOfferExpiresAt": null,
  "attended": null,
  "personalNote": null,
  "requestedAt": "2026-10-25T20:15:00Z",
  "respondedAt": null
}
```

groups/{groupId}

```
{
  "name": "West Side Pickup Crew",
  "nameLowercase": "west side pickup crew",
  "description": "Friendly weeknight soccer. Newcomers welcome.",
  "photoURL": null,
  "sports": ["soccer"],
  "localArea": "Oakland / CMU",
  "visibility": "public",
  "membershipType": "open",
  "ownerId": "u_jamie",
  "memberCount": 48,
  "conversationId": "c_grp_westside",
  "createdAt": "2026-09-10T00:00:00Z"
}
```

groups/{groupId}/members/{userId} — `role` is `owner`, `admin`, `member`, or `pending`

```
{
  "displayName": "Alex Morgan",
  "role": "member",
  "joinedAt": "2026-10-01T19:00:00Z"
}
```

conversations/{conversationId} — one collection for direct, game, and group chats

```
{
  "type": "game",
  "title": "Pickup Soccer",
  "gameId": "gm_7d2k",
  "groupId": null,
  "participantIds": ["u_jamie", "u_alex", "u_mina"],
  "lastMessageText": "Thanks! Field is the lower turf now.",
  "lastMessageAt": "2026-10-28T22:24:00Z",
  "unreadCounts": { "u_alex": 0, "u_mina": 2 }
}
```

conversations/{conversationId}/messages/{messageId}

```
{
  "senderId": "u_mina",
  "senderName": "Mina Kim",
  "text": "Can someone bring an extra ball?",
  "mediaURL": null,
  "sentAt": "2026-10-28T22:12:00Z"
}
```

reviews/{reviewId}

```
{
  "gameId": "gm_7d2k",
  "reviewerId": "u_alex",
  "revieweeId": "u_jamie",
  "isMVP": true,
  "playAgain": true,
  "tags": ["reliable", "goodSport"],
  "comment": "Clear communication and made sure everyone was involved.",
  "createdAt": "2026-10-29T01:05:00Z"
}
```

`reports/{reportId}` — `status` is `open`, `reviewing`, or `resolved`

```
{
  "reporterId": "u_alex",
  "reportedUserId": "u_83jd",
  "gameId": "gm_7d2k",
  "reason": "noShow",
  "details": "RSVP'd to three games and didn't come to any.",
  "status": "open",
  "createdAt": "2026-10-29T02:00:00Z"
}
```

Firebase Storage paths (only the download URL is saved in Firestore):

Path	Saved in
`users/{userId}/avatar.jpg`	`users/{userId}.photoURL`
`games/{gameId}/media/{fileId}`	`games/{gameId}.mediaURLs[]`
`groups/{groupId}/photo.jpg`	`groups/{groupId}.photoURL`
`conversations/{conversationId}/{fileId}`	`messages/{messageId}.mediaURL`

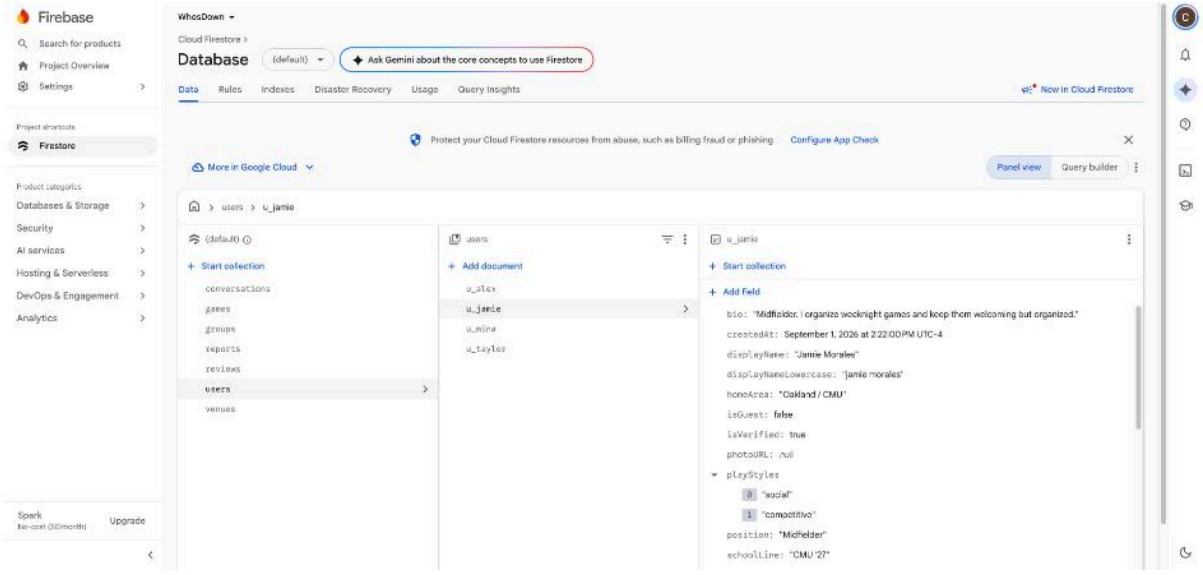
2.3 Proof that the database is set up

We built a small test app, **WhosDownDataCheck**, that holds 23 sample documents covering every collection above (4 users, settings, a friend, a notification, 2 venues, 2 games, 3 RSVPs, a group, 2 members, 2 conversations, 2 messages, a review, and a report).

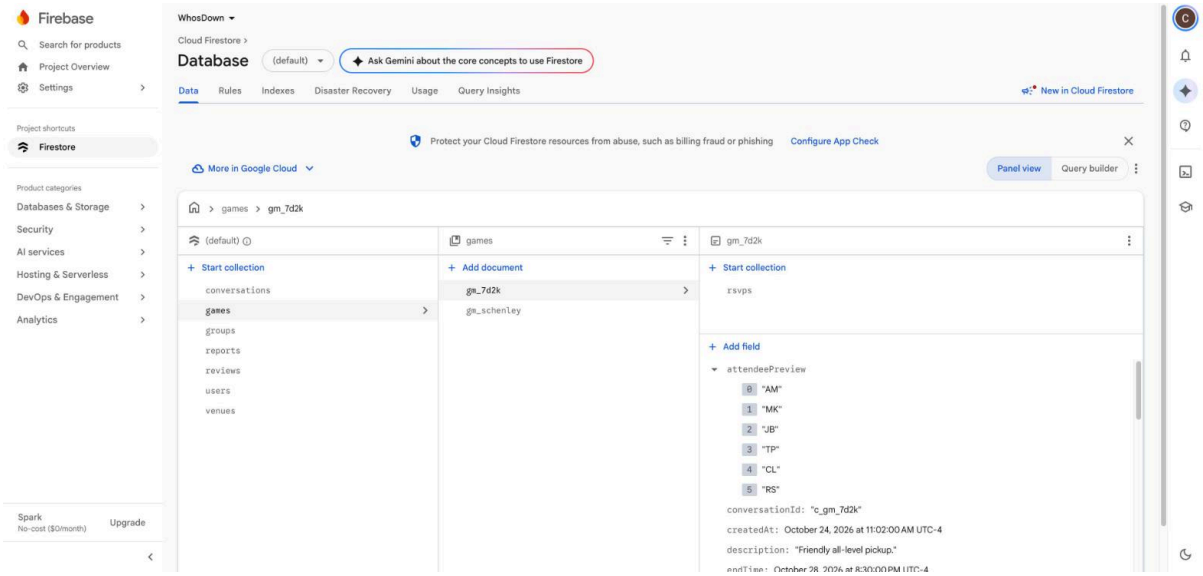
- **Models tab:** all **23 / 23** documents, including Firestore `Timestamp` and `GeoPoint` values, decode into the Swift structs in Section 3 with `Firestore.Decoder`. This proves the JSON and the structs match.
- **Firestore tab:** "Seed sample data" writes all 23 documents to our Firebase project in one batch, and "Load from Firestore" reads every collection back with `document.data(as:)`.

Live Firebase project. The sample data is in our Firebase project **WhosDown** (`whosdown-57e99`, Cloud Firestore in test mode). The Seed button wrote all 23 documents, and the app read them back.

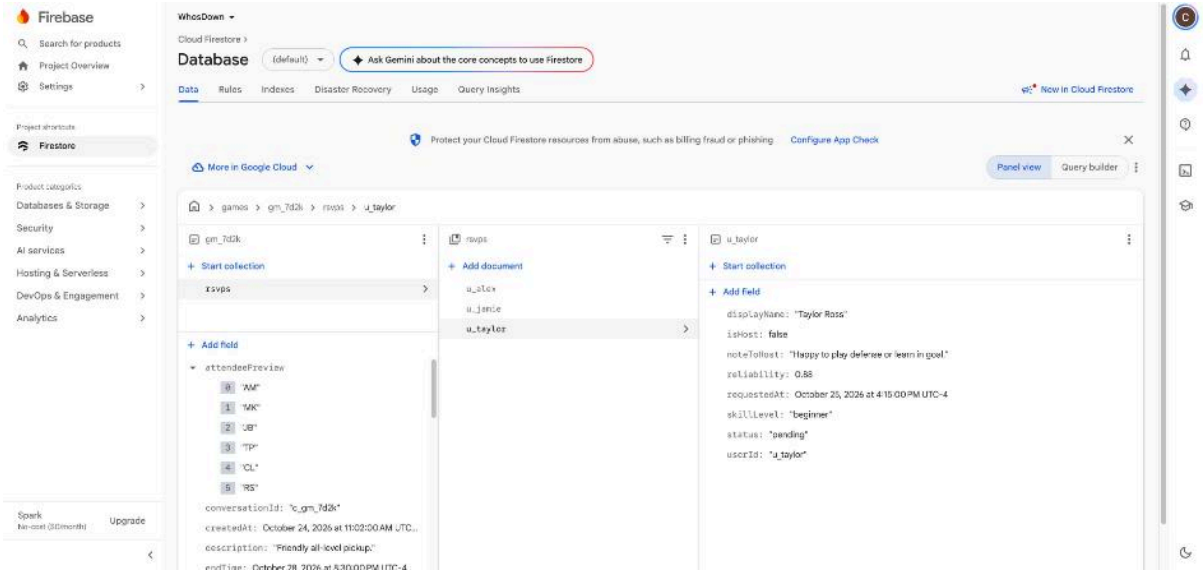
The screenshots below show the populated database in the Firebase console and our test app decoding and reading it back. Our supervising TA (atorresv@andrew.cmu.edu) also has Viewer access to the Firebase project to check the data directly.



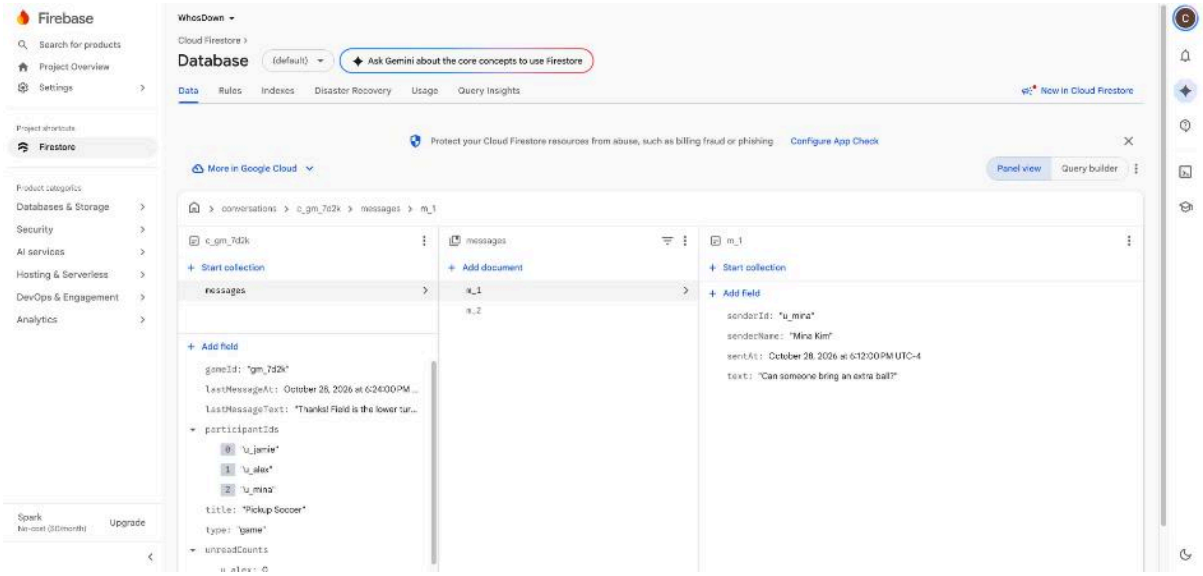
Firebase console: the users collection with a document open



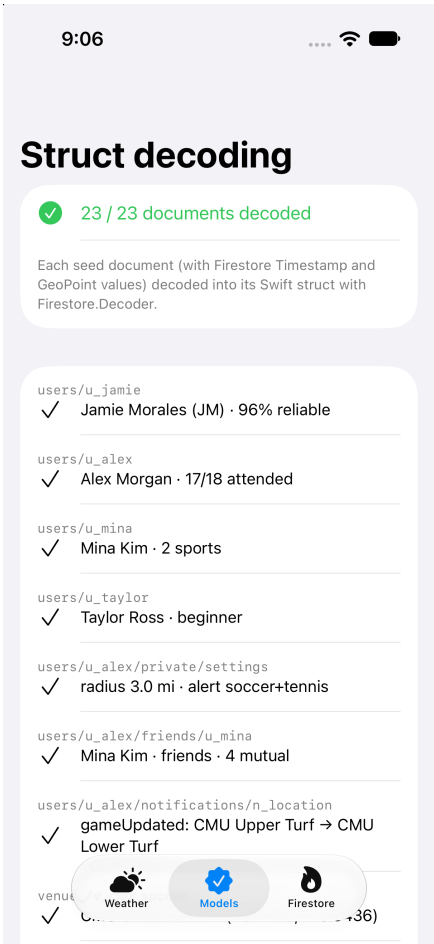
Firebase console: all 7 top-level collections, with game gm_7d2k open



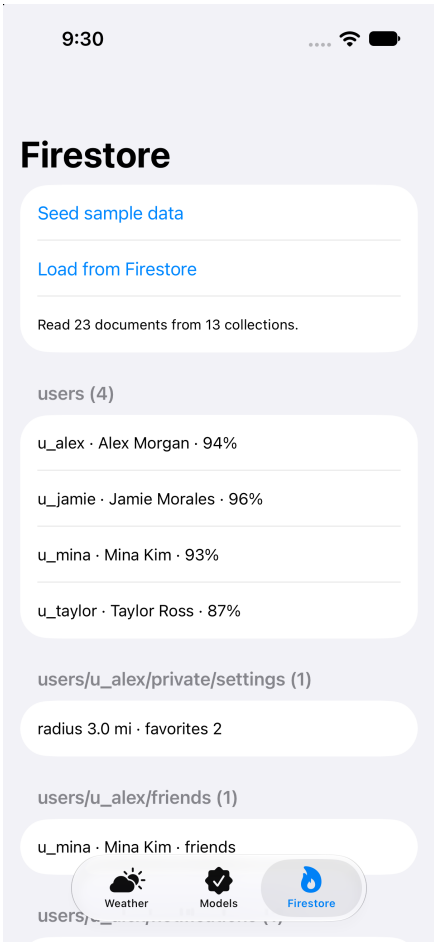
Firebase console: the games/gm_7d2k/rsvps subcollection



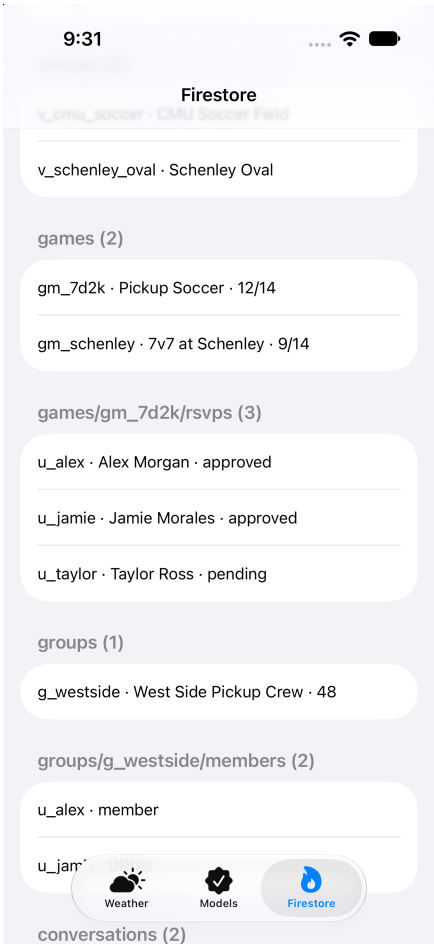
Firebase console: the conversations/c_gm_7d2k/messages subcollection



Test app, Models tab: 23 / 23 sample documents decoded into the Swift structs



Test app, Firestore tab: 23 documents read back from 13 collections



Test app, Firestore tab (scrolled): venues, games, RSVPs, groups, and members read back

3. Swift Structs

Each struct matches one Firestore document type and decodes directly with `document.data(as:)`. Computed properties (`initials`, `reliability`, `isFree`, `isFull`, `spotsLeft`, `sportName`) give the screens values they need without storing them twice. The Open-Meteo structs are in Section 4.

```
import Foundation
import FirebaseFirestore

// WhosDown app models. Each struct matches one Firestore document type
// and decodes directly with `document.data(as:)`.

// MARK: - Shared enums

enum Sport: String, Codable, CaseIterable {
  case soccer, basketball, tennis, volleyball, pickleball, baseball, football, ultimate, other
}

enum SkillLevel: String, Codable, CaseIterable {
  case beginner, casual, allLevels, intermediate, competitive
}

enum PlayStyle: String, Codable, CaseIterable {
  case social, competitive, learning
```

```
}

enum Visibility: String, Codable {
  case `public`, `private`
}

enum JoinType: String, Codable {
  case open, approvalRequired
}

enum GameStatus: String, Codable {
  case scheduled, cancelled, completed
}

enum RSVPStatus: String, Codable {
  case pending, approved, declined, waitlisted, left, removed
}

enum AudienceSetting: String, Codable {
  case everyone, whosDownPlayers, friendsAndTeammates, friends, nobody
}

enum FeedbackTag: String, Codable, CaseIterable {
  case reliable, goodSport, communicative, welcoming, competitive, helpful
}

// MARK: - users/{userId}

struct AppUser: Codable, Identifiable {
  @DocumentID var id: String?
  var displayName: String
  var displayNameLowercase: String
  var photoURL: String?
  var bio: String
  var homeArea: String
  var schoolLine: String?
  var position: String?
  var isVerified: Bool
  var isGuest: Bool
  var playStyles: [PlayStyle]
  var sportLevels: [String: SkillLevel]
  var stats: PlayerStats
  var createdAt: Date

  var initials: String {
    displayName.split(separator: " ").prefix(2).compactMap(\.first).map(String.init).joined()
  }
}

struct PlayerStats: Codable {
  var gamesRSVPd: Int
  var gamesAttended: Int
  var gamesHosted: Int
  var mvpCount: Int
  var playAgainYes: Int
  var playAgainTotal: Int
  var feedbackTagCounts: [String: Int]
  var recentFeedback: [String]
  var avgResponseMinutes: Int?
```

```
  var reliability: Double? {
    gamesRSVPd == 0 ? nil : Double(gamesAttended) / Double(gamesRSVPd)
  }
}

// MARK: - users/{userId}/private/settings

struct UserSettings: Codable {
  var email: String
  var notificationPrefs: NotificationPrefs
  var newGameAlert: NewGameAlertRule?
  var privacy: PrivacySettings
  var blockedUserIds: [String]
  var favoriteVenueIds: [String]
  var pinnedGroupIds: [String]
  var searchRadiusMiles: Double
  var fcmTokens: [String]
}

struct NotificationPrefs: Codable {
  var approvals: Bool
  var timeOrLocationChanges: Bool
  var cancellations: Bool
  var reviewReminders: Bool
  var newNearbyGames: Bool
  var groupActivity: Bool
  var messages: Bool
}

struct NewGameAlertRule: Codable {
  var sports: [Sport]
  var radiusMiles: Double
  var weekdaysOnly: Bool
  var earliestHour: Int
  var skillLevels: [SkillLevel]
}

struct PrivacySettings: Codable {
  var gameActivity: AudienceSetting
  var profileVisibility: AudienceSetting
  var directMessages: AudienceSetting
  var friendRequests: AudienceSetting
}

// MARK: - users/{userId}/friends/{friendId}

enum FriendStatus: String, Codable {
  case requestSent, requestReceived, friends
}

struct Friendship: Codable, Identifiable {
  @DocumentID var id: String?
  var displayName: String
  var status: FriendStatus
  var mutualCount: Int
  var since: Date
}
```

```
// MARK: - users/{userId}/notifications/{notificationId}

enum NotificationType: String, Codable {
  case approved, declined, gameUpdated, gameCancelled, spotOpen, message, groupActivity,
reviewReminder, newGame
}

struct AppNotification: Codable, Identifiable {
  @DocumentID var id: String?
  var type: NotificationType
  var title: String
  var body: String
  var gameId: String?
  var groupId: String?
  var conversationId: String?
  var change: GameChange?
  var expiresAt: Date?
  var isRead: Bool
  var createdAt: Date
}

struct GameChange: Codable {
  var field: String
  var oldValue: String
  var newValue: String
  var hostNote: String?
}

// MARK: - venues/{venueId}

struct Venue: Codable, Identifiable {
  @DocumentID var id: String?
  var name: String
  var address: String
  var location: GeoPoint
  var geohash: String
  var isIndoor: Bool
  var surface: String?
  var hasLights: Bool
  var sports: [Sport]
  var createdBy: String
}

// MARK: - games/{gameId}

struct Game: Codable, Identifiable {
  @DocumentID var id: String?
  var title: String
  var titleLowercase: String
  var sport: Sport
  var customSportName: String?
  var description: String
  var notes: String?
  var hostId: String
  var hostName: String
  var groupId: String?

  var venueId: String
  var venueName: String
```

```
  var location: GeoPoint
  var geohash: String
  var isIndoor: Bool
  var surface: String?
  var hasLights: Bool

  var startTime: Date
  var endTime: Date

  var skillLevel: SkillLevel
  var visibility: Visibility
  var joinType: JoinType
  var priceCents: Int
  var playerLimit: Int
  var waitlistEnabled: Bool

  var joinedCount: Int
  var pendingCount: Int
  var waitlistCount: Int
  var attendeePreview: [String]

  var fieldReserved: Bool
  var reservationWindow: String?
  var facilityNote: String?
  var mediaURLs: [String]

  var status: GameStatus
  var cancelReason: String?
  var conversationId: String
  var createdAt: Date
  var updatedAt: Date

  var isFree: Bool { priceCents == 0 }
  var isFull: Bool { joinedCount >= playerLimit }
  var spotsLeft: Int { max(playerLimit - joinedCount, 0) }
  var sportName: String { customSportName ?? sport.rawValue.capitalized }
}

// MARK: - games/{gameId}/rsvps/{userId}

struct RSVP: Codable, Identifiable {
  @DocumentID var id: String?
  var userId: String
  var displayName: String
  var reliability: Double?
  var skillLevel: SkillLevel?
  var status: RSVPStatus
  var isHost: Bool
  var noteToHost: String?
  var hostResponse: String?
  var waitlistPosition: Int?
  var spotOfferExpiresAt: Date?
  var attended: Bool?
  var personalNote: String?
  var requestedAt: Date
  var respondedAt: Date?
}

// MARK: - groups/{groupId}
```

```
enum MembershipType: String, Codable {
  case open, requestToJoin
}

struct PlayGroup: Codable, Identifiable {
  @DocumentID var id: String?
  var name: String
  var nameLowercase: String
  var description: String
  var photoURL: String?
  var sports: [Sport]
  var localArea: String
  var visibility: Visibility
  var membershipType: MembershipType
  var ownerId: String
  var memberCount: Int
  var conversationId: String
  var createdAt: Date
}

// MARK: - groups/{groupId}/members/{userId}

enum GroupRole: String, Codable {
  case owner, admin, member, pending
}

struct GroupMember: Codable, Identifiable {
  @DocumentID var id: String?
  var displayName: String
  var role: GroupRole
  var joinedAt: Date
}

// MARK: - conversations/{conversationId}

enum ConversationType: String, Codable {
  case direct, game, group
}

struct Conversation: Codable, Identifiable {
  @DocumentID var id: String?
  var type: ConversationType
  var title: String
  var gameId: String?
  var groupId: String?
  var participantIds: [String]
  var lastMessageText: String?
  var lastMessageAt: Date?
  var unreadCounts: [String: Int]
}

// MARK: - conversations/{conversationId}/messages/{messageId}

struct Message: Codable, Identifiable {
  @DocumentID var id: String?
  var senderId: String
  var senderName: String
  var text: String
```

```
  var mediaURL: String?
  var sentAt: Date
}

// MARK: - reviews/{reviewId}

struct Review: Codable, Identifiable {
  @DocumentID var id: String?
  var gameId: String
  var reviewerId: String
  var revieweeId: String
  var isMVP: Bool
  var playAgain: Bool
  var tags: [FeedbackTag]
  var comment: String?
  var createdAt: Date
}

// MARK: - reports/{reportId}

enum ReportReason: String, Codable {
  case harassment, noShow, misleadingEvent, other
}

enum ReportStatus: String, Codable {
  case open, reviewing, resolved
}

struct Report: Codable, Identifiable {
  @DocumentID var id: String?
  var reporterId: String
  var reportedUserId: String
  var gameId: String?
  var reason: ReportReason
  var details: String?
  var status: ReportStatus
  var createdAt: Date
}
```

4. External API: Open-Meteo Weather

4.1 Why we use it

In our Sprint 1 interviews, field conditions and uncertainty were the top pain point ("playing pickup is a bet on that day's session"). For outdoor games, the app shows a weather line such as **"64°F · Clear · 8% rain · Outdoor"** on Game Detail (H09), Joined Game (H14), Venue & Details (H18), and Review & Publish (H19). The line is hidden when `Game.isIndoor == true`.

Open-Meteo is free for non-commercial use and **needs no API key**, so the playground runs as-is.

4.2 Request

```
GET https://api.open-meteo.com/v1/forecast
    ?latitude=40.4433
    &longitude=-79.9436
    &hourly=temperature_2m,precipitation_probability,weather_code,wind_speed_10m
    &temperature_unit=fahrenheit
    &wind_speed_unit=mph
    &timezone=America/New_York
```

```
    &forecast_days=7
```

`latitude` and `longitude` come from `Game.location` (a Firestore `GeoPoint`). Seven days covers any game a user can see on Discover.

4.3 Response (shortened — the real response has 168 hourly rows)


```
{
  "latitude": 40.4349,
  "longitude": -79.93617,
  "utc_offset_seconds": -14400,
  "timezone": "America/New_York",
  "hourly_units": {
    "time": "iso8601",
    "temperature_2m": "°F",
    "precipitation_probability": "%",
    "weather_code": "wmo code",
    "wind_speed_10m": "mp/h"
  },
  "hourly": {
    "time": ["2026-10-25T00:00", "2026-10-25T01:00"],
    "temperature_2m": [46.9, 44.9],
    "precipitation_probability": [0, 0],
    "weather_code": [0, 0],
    "wind_speed_10m": [4.8, 2.9]
  }
}
```


The response uses **parallel arrays**: index *i* of every array is the same hour. The app turns them into one `HourlyForecast` per hour, then picks the hour closest to `Game.startTime`.

4.4 Structs (same code as the playground)

```
// MARK: - Decodable structs for the Open-Meteo response

struct ForecastResponse: Decodable {
  let latitude: Double
  let longitude: Double
  let timezone: String
  let utcOffsetSeconds: Int
  let hourlyUnits: HourlyUnits
  let hourly: Hourly

  enum CodingKeys: String, CodingKey {
    case latitude, longitude, timezone
    case utcOffsetSeconds = "utc_offset_seconds"
    case hourlyUnits = "hourly_units"
    case hourly
  }
}

struct HourlyUnits: Decodable {
  let temperature: String
  let windSpeed: String

  enum CodingKeys: String, CodingKey {
    case temperature = "temperature_2m"
    case windSpeed = "wind_speed_10m"
  }
}
```

```
// Open-Meteo returns parallel arrays: index i of every array is the same hour.
struct Hourly: Decodable {
  let time: [String]
  let temperature: [Double]
  let precipitationProbability: [Int]
  let weatherCode: [Int]
  let windSpeed: [Double]

  enum CodingKeys: String, CodingKey {
    case time
    case temperature = "temperature_2m"
    case precipitationProbability = "precipitation_probability"
    case weatherCode = "weather_code"
    case windSpeed = "wind_speed_10m"
  }
}

// MARK: - App-side struct the interface reads

struct HourlyForecast {
  let date: Date
  let temperatureF: Double
  let precipitationProbability: Int
  let weatherCode: Int
  let windSpeedMph: Double

  // WMO weather codes used by Open-Meteo
  var conditionText: String {
    switch weatherCode {
    case 0: return "Clear"
    case 1: return "Mostly clear"
    case 2: return "Partly cloudy"
    case 3: return "Overcast"
    case 45, 48: return "Fog"
    case 51, 53, 55, 56, 57: return "Drizzle"
    case 61, 63, 65, 66, 67: return "Rain"
    case 71, 73, 75, 77: return "Snow"
    case 80, 81, 82: return "Rain showers"
    case 85, 86: return "Snow showers"
    case 95, 96, 99: return "Thunderstorm"
    default: return "Unknown"
    }
  }

  // Line shown on H09 / H14 / H18, e.g. "64°F · Clear · 8% rain"
  var summaryLine: String {
    "\(Int(temperatureF.rounded()))°F · \(conditionText) · \(precipitationProbability)% rain"
  }
}

extension ForecastResponse {
  // Turns the parallel arrays into one HourlyForecast per hour.
  // Times come back in local time without an offset, so the response's
  // utc_offset_seconds is applied when parsing.
  var hours: [HourlyForecast] {
    let formatter = DateFormatter()
    formatter.locale = Locale(identifier: "en_US_POSIX")
    formatter.dateFormat = "yyyy-MM-dd'T'HH:mm"
```

```
    formatter.timeZone = TimeZone(secondsFromGMT: utcOffsetSeconds)

    return hourly.time.indices.compactMap { i in
      guard let date = formatter.date(from: hourly.time[i]) else { return nil }
      return HourlyForecast(
        date: date,
        temperatureF: hourly.temperature[i],
        precipitationProbability: hourly.precipitationProbability[i],
        weatherCode: hourly.weatherCode[i],
        windSpeedMph: hourly.windSpeed[i]
      )
    }
  }

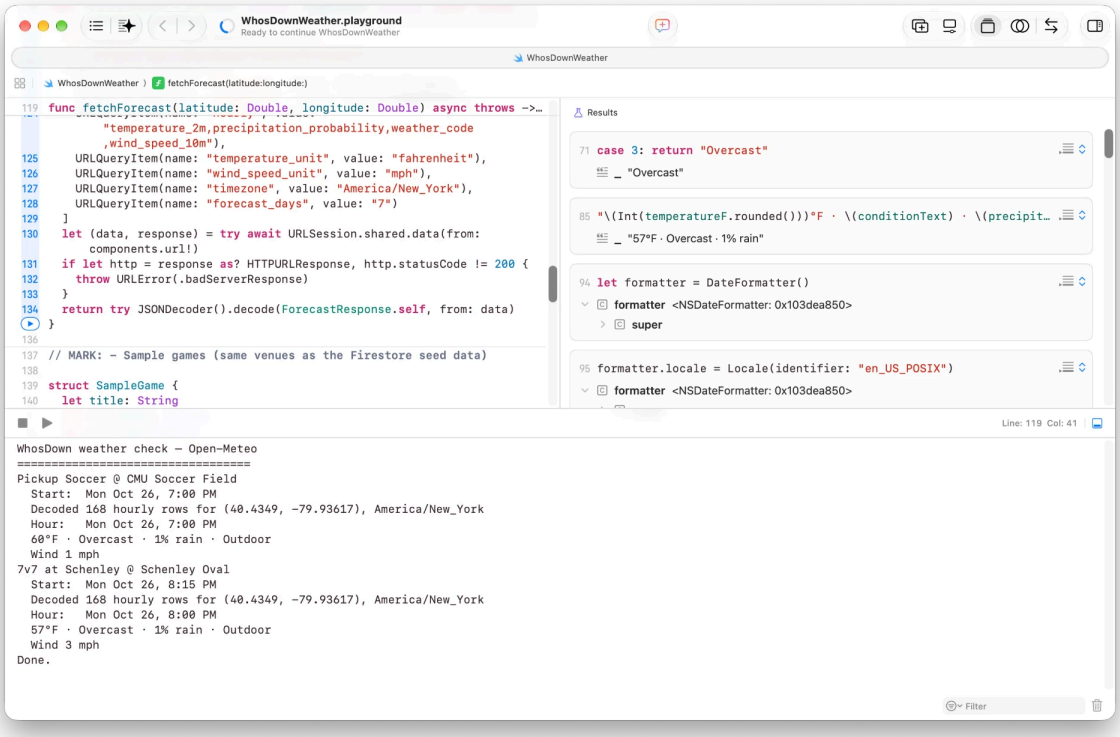
  // The hour closest to a game's start time.
  func forecast(closestTo date: Date) -> HourlyForecast? {
    hours.min { abs($0.date.timeIntervalSince(date)) < abs($1.date.timeIntervalSince(date)) }
  }
}
```

4.5 Playground (`WhosDownWeather.playground`, submitted with this document)

The playground uses `URLSession` with `async/await` to request the forecast for two sample games (the same venues as our Firestore sample data), decodes the response with `JSONDecoder` into `ForecastResponse`, and prints the hour closest to each game's start time. Open it in Xcode and press Run. It also runs from Terminal with `swift Contents.swift`.

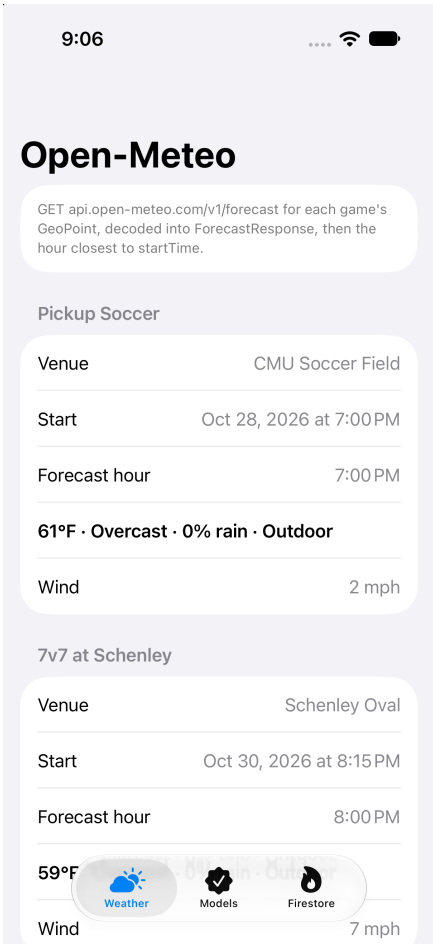
Output (run on Oct 25, 2026):

```
WhosDown weather check — Open-Meteo
==================================
Pickup Soccer @ CMU Soccer Field
  Start:  Mon Oct 26, 7:00 PM
  Decoded 168 hourly rows for (40.4349, -79.93617), America/New_York
  Hour:   Mon Oct 26, 7:00 PM
  60°F · Overcast · 1% rain · Outdoor
  Wind 1 mph
7v7 at Schenley @ Schenley Oval
  Start:  Mon Oct 26, 8:15 PM
  Decoded 168 hourly rows for (40.4349, -79.93617), America/New_York
  Hour:   Mon Oct 26, 8:00 PM
  57°F · Overcast · 1% rain · Outdoor
  Wind 3 mph
Done.
```



The playground run in Xcode, console output at the bottom

The same code also runs inside our test app (below), where each sample `Game` decoded from Firestore data is passed to `fetchForecast` with its `GeoPoint`.



Open-Meteo data decoded in the test app

4.6 Apple MapKit and Core Location

MapKit is an on-device framework, not a JSON API, so it needs no playground. It supplies:

Data	API	Screens
Map, pins, radius circle	SwiftUI `Map`, `Annotation`, `MapCircle`	H06, H40
Place / address search	`MKLocalSearch` → `MKMapItem` (name, address, coordinate) saved as a new `Venue`	H40
User location	`CLLocationManager` (when-in-use)	H05, H06, H07
Distance to a game or venue	`CLLocation.distance(from:)`, shown in miles	H06, H07, H09, H23, H40, H53, H56

5. Internal API and On-Device Store

- **Internal API: not applicable.** We use Firebase instead of a custom server (Section 1). All reads and writes go through the Firebase iOS SDK, and every data source is listed in Section 8.2.
- **SwiftData: not applicable.** WhosDown does not keep its own on-device database. Firestore's built-in offline cache keeps recently loaded games, RSVPs, and messages

available without a connection and syncs writes when the device reconnects. Small view preferences (for example, the last-used Discover filters) use `@AppStorage`.

6. Interactive Prototype

Figma prototype: https://www.figma.com/proto/4dTTJVikeBUzFTG9KnSowN/Who-sDown-%E2%80%94-Lo-fi-Wireframes?node-id=104-23&m=dev&scaling=min-zoom&content-scaling=fixed&page-id=104%3A16&starting-point-node-id=104%3A23

The prototype is built from our 63 high-fidelity screens (H01–H63, Figma page "03 — Hi-Fi Full Marks"). It looks like the real app and clicks through to the right screens, but it does not save data. The tab bar (Discover, My Games, Create, Social, Profile) works from every main screen.

Flows the prototype covers

Flow	Screens
Sign up and onboarding	H01 Welcome → H02 Create Account → H03 Select Sports → H42 Skill Levels → H04 Complete Profile → H05 Permissions → H43 Notification Permission → H44 Onboarding Complete
Sign in / guest	H01 → H41 Sign In; H01 → "Explore as guest" → H06; guest taps Join → H34 Guest Join Gate
Find a game	H06 Discover Map ↔ H07 Discover List → H08 Filters → H09 Game Detail → H10 Organizer Profile, H11 Player Roster; H48 Search
Join a game (approval)	H09 → H12 Join Review → H13 Pending → H14 Joined Game (approved) or H35 Declined
Full game / waitlist	H09 → H15 Full / Waitlist → H53 Waitlist Spot Available → claim spot
Coordinate	H14 → H16 Event Chat; H14 → H37 Leave Confirmation
Host a game	H17 Create Activity → H40 Choose Location → H18 Venue & Details → H19 Review & Publish → H45 Game Created
Manage a game	H20 Host Dashboard → H21 Join Requests, H46 Edit Game, H22 Transfer Host, H38 Cancel Game
Social	H23 Friends Activity, H24 Direct Messages → H39 Message Thread; H25 Groups Discover → H26 Group Detail; H47 Create Group
History and reputation	H27 My Games → H28 Past Games → H29 Past Game Summary → H30 Post-game Review; H60 Review Players → H61 Player Review Detail → H62 Review Summary (H63 changed state)
Notifications	H31 Notifications → H50 All Notifications → H51 Game Update, H52 Game Cancelled, H53 Spot Available, H54 Message, H55 Group Activity

Flow	Screens
Profile and settings	H36 Profile & Preferences → H49 Edit Profile, H32 Notification Settings, H56 Favorite Fields, H57 Privacy Settings, H58 Reliability & Reviews, H59 Account & Security; H33 Safety & Reporting

7. User Testing Report

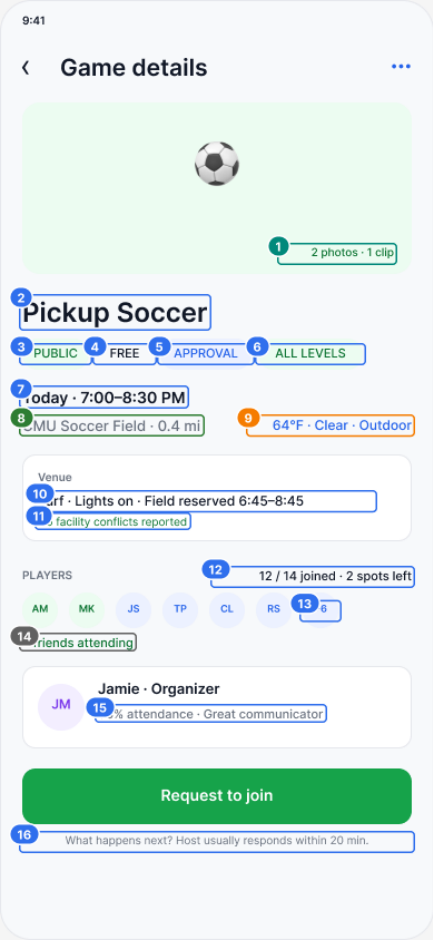
The full report is a separate document, **WhosDown (Team 6) Sprint 3 - User Interviews Report**. It covers the method, the 4 participants, what worked, 23 lessons learned with the change each one causes, and the full interview notes. In short:

- Testers trust the app more when they know **who the host is, what the game costs, and who else is going**. Safety came up strongly (P4), so **reporting moves into V1** and host reputation stays a top V2 item.
- Testers expect the app to **remind them**. A local reminder before each approved game is **added to V1**, because it needs no server. Push alerts stay in V2.
- The biggest requests are **social and group features**: friends attending, mutual friends, group profiles with schedules, sharing, and favorites. They confirm our V2 plan and add six V2 items (2.18–2.23).
- Watching testers use the screens showed **clarity problems in the core loop**: the organizer profile was hard to find, testers did not understand approval, leaving, or what happens when a game fills up or ends, and statuses all looked alike. These fixes go into **V1** (lessons 16–20, 22), together with a layout pass (1.26).
- Testers liked the clean interface, the map, how easy it was to post a game, private games, and the roster. No tester asked to remove a feature, and the core loop (find, join, host) stays the same.

8. Wireframe Data Mapping

This section maps every piece of data on the 63 high-fidelity screens to where it comes from (8.2) and which field or Swift property holds it (8.3). Static text (headings, helper text, button labels) is not marked.

Marked-up wireframes (Figma): page **"05 — Sprint 3 Data Mapping"** in our Figma file (https://www.figma.com/design/4dTTJVikeBUzFTG9KnSowN). It holds a copy of each of the 63 hi-fi screens. Every piece of data on a screen has a numbered, color-coded badge (Firestore, Auth, Storage, Open-Meteo, MapKit, user input, computed). The panel beside each screen lists each badge's source and field, the endpoint IDs from 8.2 that the screen uses, and its release (V1 or V2 from Section 9). An endpoint legend sits to the right of the first row. All 63 marked-up screens are in the separate Mapped Wireframes PDF (8.3).



Figma example: H09 Game Detail with numbered data badges

The panel beside H09 in Figma lists each numbered badge:

#	Source	Field
1	Storage	`games.mediaURLs` (Storage `games/{id}/media`)
2	Firestore	`games/{id}.title`
3	Firestore	`games.visibility`
4	Firestore	`games.priceCents` (`Game.isFree`)
5	Firestore	`games.joinType`
6	Firestore	`games.skillLevel`
7	Firestore	`games.startTime` / `endTime`
8	MapKit	`games.venueName` + distance to `games.location`
9	Open-Meteo	Hour closest to `startTime` (`HourlyForecast`) + `games.isIndoor`
10	Firestore	`games.surface`, `hasLights`, `fieldReserved`, `reservationWindow`
11	Firestore	`games.facilityNote`
12	Firestore	`games.joinedCount`, `playerLimit`, `spotsLeft`

#	Source	Field
13	Firestore	`games.attendeePreview`
14	Computed	`games/{id}/rsvps` (approved) ∩ `users/{me}/friends`
15	Firestore	`users/{hostId}.stats` (`PlayerStats.reliability`, `feedbackTagCounts`)
16	Firestore	`users/{hostId}.stats.avgResponseMinutes`

Endpoints: R2, R3, R4, R7, R9 · **Other sources:** Storage, Open-Meteo, MapKit · **Release:** V1

8.1 Source key

Tag	Meaning
FS	Cloud Firestore document or query (path shown)
Auth	Firebase Authentication (`Auth.auth().currentUser`)
Storage	Firebase Storage (photos and clips; download URL saved in Firestore)
FCM	Firebase Cloud Messaging push
OM	Open-Meteo forecast API (Section 4)
MK	Apple MapKit / Core Location (map, place search, user location, distance)
Input	Entered by the user on this screen, then written to the FS path shown
Computed	Derived on the device from the fields listed
Local	View state only, not stored

8.2 Data sources ("endpoints")

Because we use Firebase, our "endpoints" are the Firestore queries and writes the app makes. Every screen in the marked-up wireframes (8.3) uses one or more of these.

Firestore reads

ID	Name	Path and query	Type	Screens
R1	Nearby games	`games` where `geohash` in ranges covering the radius, `status == "scheduled"`, `visibility == "public"`, `startTime >= now`; then exact distance (MK) and filters on device	get	H06, H07, H08, H15, H35, H52
R2	One game	`games/{gameId}`	listener	H09, H11–H15, H19–H22, H29, H34, H37, H38, H45, H46, H51–H53, H55
R3	Game roster	`games/{gameId}/rsvps` where `status == "approved"` (or `"pending"` for the host)	listener	H09, H11, H21, H22, H29, H30, H60
R4	My RSVP	`games/{gameId}/rsvps/{me}`	listener	H09, H12–H15, H29, H35, H37, H53
R5	My RSVPs	collection group `rsvps` where `userId == me`; parent `games` fetched by ID	get	H06 "Next up", H27, H28

ID	Name	Path and query	Type	Screens
R6	Games I host	`games` where `hostId == me`, `status == "scheduled"`	listener	H20, H27
R7	User profile	`users/{userId}`	get	H09 organizer, H10, H11, H29, H36, H39, H49, H58
R8	My settings	`users/{me}/private/settings`	listener	H05, H08, H32, H33, H36, H56, H57, H59
R9	Friends	`users/{me}/friends` where `status == "friends"`	listener	H09, H11, H23, H24, H25, H29
R10	Notifications	`users/{me}/notifications` order by `createdAt` desc	listener	H31, H50–H55
R11	Venues	`venues` where `geohash` in ranges, plus favorites by ID	get	H18, H40, H56
R12	My conversations	`conversations` where `participantIds` array-contains `me`, order by `lastMessageAt` desc	listener	H20, H24
R13	Messages	`conversations/{id}/messages` order by `sentAt`	listener	H16, H39, H54
R14	Groups	`groups` where `visibility == "public"`, `sports` array-contains-any my sports; `groups/{groupId}`	get	H25, H26, H55
R15	Group members	`groups/{groupId}/members`	get	H25, H26
R16	Group games	`games` where `groupId == id`, `startTime >= now` (count query for "games this week")	get	H25, H26, H48
R17	Friends activity	collection group `rsvps` where `userId in friendIds`, `status == "approved"`; collection group `members` where friend ID, recent `joinedAt`	get	H23
R18	My reviews	`reviews` where `reviewerId == me`	get	H28
R19	Search	prefix queries on `games.titleLowercase`, `groups.nameLowercase`, `users.displayNameLowercase`	get	H48

Firestore writes

ID	Action	What is written	Screens
W1	Create account	`users/{uid}` + `users/{uid}/private/settings`	H02–H05, H42–H44
W2	Edit profile	update `users/{me}`; photo to Storage	H04, H49
W3	Publish game	batch: new `games/{id}` + `rsvps/{me}` (host, approved) + `conversations/{id}` (type `game`)	H19

ID	Action	What is written	Screens
W4	Edit game	update `games/{id}` + `gameUpdated` notification to each approved player	H46
W5	Cancel game	`games/{id}.status = cancelled`, `cancelReason` + `gameCancelled` notification to every RSVP	H38
W6	Join / request / waitlist	transaction: `rsvps/{me}` + `joinedCount`, `pendingCount`, or `waitlistCount`	H12, H15
W7	Approve / decline	transaction: `rsvps/{userId}.status`, `respondedAt`, `hostResponse` + counters + notification	H21
W8	Leave / cancel request	transaction: `rsvps/{me}.status = left` + counters; first waitlisted RSVP gets `spotOfferExpiresAt` + `spotOpen` notification	H13, H37
W9	Claim spot	transaction: `rsvps/{me}.status = approved`, `joinedCount + 1`, `waitlistCount − 1`	H53
W10	Transfer host	`games/{id}.hostId`, `hostName`; swap `isHost` on two RSVPs	H22
W11	Send message	new `messages/{id}` + `lastMessageText`, `lastMessageAt`, `unreadCounts` on the conversation	H16, H39
W12	Friend request / accept	`users/{me}/friends/{id}` and `users/{id}/friends/{me}`	H10, H23, H29
W13	Create / join group	new `groups/{id}` + `members/{me}` + `conversations` (type `group`); or `members/{me}` + `memberCount + 1`	H26, H47
W14	Submit reviews	one `reviews/{id}` per teammate + reviewee `stats` incremented	H30, H60–H62
W15	Report / block	new `reports/{id}`; or `blockedUserIds` in settings	H33, H57
W16	Save settings	update `users/{me}/private/settings`	H05, H08, H26, H32, H43, H56, H57
W17	Mark read	`notifications/{id}.isRead = true`; `unreadCounts[me] = 0`	H16, H24, H31, H50–H55
W18	Add venue	new `venues/{id}` from an `MKMapItem`	H40
W19	Personal note	`rsvps/{me}.personalNote`	H29

Other sources

Source	Calls	Screens
Auth	`createUser`, `signIn`, `signInAnonymously`, `link(with:)`, `sendPasswordReset`, `updatePassword`, `delete`, `providerData`	H01, H02, H34, H41, H59
Storage	upload / download URL for avatar, game media, group photo, chat photo	H04, H09, H16, H18, H47, H49
FCM	device token saved to `fcmTokens`; push sent for approvals, changes, cancellations, spot open, messages	H43, H50–H55
OM	`GET /v1/forecast` (Section 4)	H09, H14, H18, H19
MK	`Map`, `MKLocalSearch`, `CLLocationManager`, `distance(from:)`	H05, H06, H07, H09, H23, H40, H53, H56
EventKit	add game to calendar	H14

8.3 Screen-by-screen mapping (marked-up wireframes)

The full screen-by-screen mapping is the set of marked-up wireframes itself:

- **PDF (submitted with this document):** *WhosDown (Team 6) Sprint 3 - Mapped Wireframes.pdf*. It shows all 63 screens, two per page, each next to its source panel. The color key is in 8.1 and the endpoint IDs are in 8.2.
- **Live Figma file:** page **"05 — Sprint 3 Data Mapping"** at https://www.figma.com/design/4dTTJVikeBUzFTG9KnSowN

For every screen, each numbered badge names its source (Firestore path, Auth, Storage, Open-Meteo, MapKit, user input, or computed) and the field or struct property it uses, and the panel lists the endpoints from 8.2 and the release (V1 or V2). H09 above shows the format.

9. Feature List and Delivery Schedule

9.1 Scope rules

- **Version 1** is the end of Sprint 4. It delivers the core loop from Sprint 1: *find a nearby game → join it → show up*, and *post a game → manage who comes*. A Version 1 feature must work end to end with real Firestore data.
- **Version 2** is the final release. It adds the social, reputation, and push features that make users come back.
- Firestore counters and in-app notifications are written by the client in transactions in Version 1. Push notifications need a server to send them, so FCM sending (through Cloud Functions) is in Version 2.
- Dates are the start of each work week. Sprint end dates follow Canvas.
- User testing (Section 7) changed the plan: V1 gained game reminders (1.24), reporting (1.25), and a layout pass (1.26); V1 1.4, 1.7, 1.8, 1.11, 1.14, 1.16, 1.17, 1.19, and 1.22 gained clarity fixes testers asked for; and V2 gained 2.18–2.23.

9.2 Version 1 (end of Sprint 4)

#	Feature	What works in V1	Screens	Week of
1.1	Project setup	Xcode project, Firebase SDK, `Models.swift`, sample data in Firestore, Firestore security rules (owner-only `private/settings`)	—	Oct 26
1.2	Account and sign-in	Email sign-up, sign-in, sign-out, password reset	H01, H02, H41	Oct 26
1.3	Guest browsing	Anonymous sign-in; guest can browse; Join opens the guest gate, which upgrades the account	H01, H34	Oct 26
1.4	Onboarding	Select from all nine sports plus "Other" with a custom name, skill level per sport, basic profile (name, bio, home area, play style), location permission, notification preferences saved	H03, H42, H04, H05, H44	Oct 26
1.5	Discover map	MapKit map with user location, radius circle, sport pins from the nearby-games query, pin card, "Next up" card	H06	Nov 2
1.6	Discover list	List of nearby games with distance, level, count, price	H07	Nov 2
1.7	Filters	Sport, time (Now / Tonight / Tomorrow / custom range), distance typed in miles, skill, join type; distance saved to settings	H08	Nov 2
1.8	Game detail	All game facts, distance, venue facts, attendee preview; whole organizer row opens the organizer profile; note that the game moves to Past Games after it ends	H09	Nov 2
1.9	Weather	Open-Meteo weather line for outdoor games	H09, H14, H18, H19	Nov 2
1.10	Player roster	Approved players with level and reliability	H11	Nov 2
1.11	Create game	Sport (including a custom "Other" sport), date/time, skill, player limit, join type, visibility, price (display only), notes, facility details	H17, H18, H19, H45	Nov 9
1.12	Choose location	MapKit place search + saved venues; new place saved as a venue	H40	Nov 9
1.13	Join an open game	One-tap join; counters update live	H09, H14	Nov 9
1.14	Request to join	Join review explains the three approval steps; note to host; Pending; live change to Approved or Declined (with host message); one color per status	H12, H13, H14, H35	Nov 9
1.15	Host: approve / decline	Host dashboard and join-request list	H20, H21	Nov 9
1.16	Leave game / cancel request	"Leave game" or "Cancel request" button on Game Detail whenever the player has an RSVP; confirmation sheet; counters update	H09, H13, H37	Nov 9

#	Feature	What works in V1	Screens	Week of
1.17	Waitlist	Full games say "Full · Join waitlist" with the waitlist count; join waitlist; first in line gets a 10-minute spot offer when someone leaves; claim spot	H15, H53	Nov 16
1.18	Edit and cancel game	Host edits time, location, limit, join type; cancels with a reason; joined players get an in-app notification	H46, H38, H51, H52	Nov 16
1.19	My Games	Upcoming (with color-coded status badge), Hosting, and Past lists; games move to Past automatically after they end	H27, H28	Nov 16
1.20	Game chat	One chat per game, created on publish; live messages for approved players	H16	Nov 16
1.21	In-app notifications	Notification list and detail screens for approved, declined, game updated, game cancelled, spot open	H31, H50–H53	Nov 16
1.22	Profile	View own profile organized by sport with a level per sport; edit profile (photo upload to Storage, name, bio, home area, sports and levels); view organizer profile	H36, H49, H10	Nov 16
1.23	Add to calendar	EventKit event from a joined game	H14	Nov 16
1.24	Game reminders *(from user testing)*	Local notification before each approved game, scheduled on the device when the RSVP is approved; removed if the player leaves or the game is cancelled	H14, H27	Nov 16
1.25	Report a user or game *(from user testing)*	Safety & Reporting form writes a `reports` document	H33	Nov 16
1.26	Layout and interaction pass *(from user testing)*	Small steps (join review, leave and cancel confirmations, approve or decline) open as bottom sheets instead of new screens; spacing and alignment on a shared 8-point grid	H09, H12, H21, H37, H38	Nov 16

9.3 Version 2 (final release)

#	Feature	What it adds	Screens	Week of
2.1	Friends	Send / accept friend requests; "friends attending" on game detail; friend tags on the roster	H10, H09, H11, H29	Nov 23
2.2	Direct messages	Conversation list, one-to-one threads, unread badges	H24, H39	Nov 23
2.3	Search	Prefix search for games, groups, and people	H48	Nov 23
2.4	Groups	Discover, detail, create (with photo), join (open or request), group chat, pin to home, group games	H25, H26, H47, H55	Nov 30
2.5	Friends activity feed	Games friends joined, groups friends joined, people you may know	H23	Nov 30

#	Feature	What it adds	Screens	Week of
2.6	Post-game reviews	Review players (MVP, play again, tags, written comment), review summary, past game summary with "new players met" and personal note; review screens explain what each answer affects	H29, H30, H60–H63	Nov 30
2.7	Reliability and reputation	Attendance marking by host, reliability %, feedback tags, MVP count, host response time	H10, H36, H58	Nov 30
2.8	Safety	Block a user, manage blocked list (reporting is in V1, 1.25)	H33, H57	Dec 7
2.9	Privacy settings	Who can see activity, profile, message me, send friend requests	H57	Dec 7
2.10	Push notifications	FCM token on device; Cloud Functions send pushes for approvals, changes, cancellations, spot open, messages, group activity, review reminders	H43, H50–H55	Dec 7
2.11	Notification settings and new-game alerts	Notification toggles; "alert me when a matching game is posted" rule	H32, H08	Dec 7
2.12	Media	Game photos and clips on game detail; photo attachments in chat	H09, H18, H16	Dec 7
2.13	Favorite fields	Star venues; favorites shown first when choosing a location	H56, H40	Dec 7
2.14	Transfer host	Pass a game to another joined player	H22	Dec 7
2.15	Invite a friend to a game	In-app invite notification	H11, H14	Dec 7
2.16	Recommendations	"Why these?" ranking by sports, distance, and play style	H07	Dec 7
2.17	Account and security	Change password, Sign in with Apple, download my data, delete account	H59	Dec 7
2.18	Profile media *(from user testing)*	Photos and media on a player's profile; needs a new `mediaURLs` field on `users`	H36, H49	Nov 30
2.19	Group schedules *(from user testing)*	Recurring events on the group profile; filter groups by sport and skill level; add recurring events to the calendar	H25, H26	Nov 30
2.20	Labeled chats *(from user testing)*	Each chat in the list shows its type and its game or group name with the sport	H24	Nov 23
2.21	Hidden location until approved *(from user testing)*	For approval games, show only the area until the player is approved; the exact location moves to a subdocument only approved players can read	H09, H13, H14	Dec 7
2.22	Share a game or group *(from user testing)*	Share sheet with a link that opens the game or group in the app	H09, H26	Nov 30
2.23	Saved games *(from user testing)*	Save a game for later; needs a new `savedGameIds` field in `private/settings`	H09, H27	Dec 7

9.4 Not planned

These were in our earlier ideas but are outside the semester scope:

- **In-app payments.** Games can show a price (`priceCents`), but payment is handled outside the app.
- **Field reservation with facilities.** Hosts enter reservation details by hand (`fieldReserved`, `reservationWindow`).
- **Coaching / tutoring marketplace.**
- **Non-sport hobbies.**

9.5 Risks to the V1 schedule

Risk	Plan
Geohash radius query is new to the team	Build it in week 1 with the sample data; fall back to a bounding-box query on latitude if needed
Counter drift (`joinedCount` vs. real RSVPs)	All join, leave, approve, and waitlist writes use one Firestore transaction (W6–W9)
Waitlist spot offer timing without a server	V1 checks `spotOfferExpiresAt` on the device when the next user opens the game; V2 moves expiry to a Cloud Function